

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل چهارم: Pattern mining: basic concepts and methods

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 4.1 Basic concepts
 - 4.1.1 Market basket analysis: a motivating example
 - 4.1.2 Frequent itemsets, closed itemsets, and association rules
- 4.2 Frequent itemset mining methods
 - 4.2.1 Apriori algorithm: finding frequent itemsets by confined candidate generation
 - 4.2.2 Generating association rules from frequent itemsets
 - 4.2.3 Improving the efficiency of Apriori
 - 4.2.4 A pattern-growth approach for mining frequent itemsets
 - 4.2.5 Mining frequent itemsets using the vertical data format
 - 4.2.6 Mining closed and max patterns
- 4.3 Which patterns are interesting?—Pattern evaluation methods
 - 4.3.1 Strong rules are not necessarily interesting
 - 4.3.2 From association analysis to correlation analysis
 - 4.3.3 A comparison of pattern evaluation measures

Section 4.1

4.1.1. Market basket analysis: a motivating example

تحلیل سبد خرید

تحلیل سبد خرید (Market Basket Analysis) یک تکنیک داده کاوی است که برای کشف ارتباطات و روابط بین آیتم‌هایی که همراه خرید می‌شوند استفاده می‌شود. هدف اصلی، شناسایی الگوهای هم‌وقوعی بین محصولات در تراکنش‌هاست تا بینشی از رفتار و ترجیحات مشتریان به دست آید. این تحلیل به طور گسترده در خرده‌فروشی، تجارت الکترونیک و سایر صنایع به کار می‌رود تا استراتژی‌های مربوط به چیدمان محصولات، تبلیغات و فروش متقابل (Cross-selling) بهینه‌سازی شوند.

مفاهیم کلیدی

- قواعد ارتباطی (Association Rules):** تحلیل سبد خرید معمولاً منجر به استخراج قواعدی به شکل «اگر مشتری کالای X را بخرد، احتمالاً کالای Y را نیز می‌خرد» می‌شود. این قواعد به درک رفتار خرید مشتریان کمک می‌کنند.
- پشتیبانی، اطمینان و لیفت (Support, Confidence, Lift):** پشتیبانی (Support) نشان می‌دهد یک مجموعه از آیتم‌ها چقدر اغلب با هم ظاهر می‌شوند. **اطمینان (Confidence)** احتمال خرید آیتم Y را وقتی آیتم X خریده شده باشد، اندازه‌گیری می‌کند.

لیفت (Lift) نشان‌دهنده قدرت ارتباط بین دو آیتم است؛ اگر بیشتر از ۱ باشد، یعنی آیتم‌ها به‌طور معناداری با هم هم‌وقوع دارند.

مثالی از تحلیل سبد خرید در پایتون

بیایید از کتابخانه **mlxtend** در پایتون استفاده کنیم تا تحلیل سبد خرید را روی یک مجموعه داده نمونه خرده‌فروشی انجام دهیم:

```
In [ ]: pip install mlxtend
```

```
In [8]: # در صورت نیاز mlxtend نصب کتابخانه
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# داده‌های نمونه: شامل تراکنش‌ها و آیتم‌ها و
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# one-hot encoding تبدیل داده‌ها به فرمت
# (وجود آیتم در تراکنش = True) هر تراکنش یک سطر، هر آیتم یک ستون
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# اجرای الگوریتم Apriori
# min_support=0.2 یعنی آیتم‌ست‌ها باید در حداقل 20% تراکنش‌ها ظاهر بشن
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# Association Rules تولید قوانین انجمنی
rules = association_rules(frequent_itemsets, metric='lift', min_threshold=1)

# برای جلوگیری از هشدار Inf و NaN پاک‌سازی: حذف مقادیر
rules = rules.replace([float('inf'), float('-inf')], pd.NA).dropna().reset_index(drop=True)

# نمایش نتایج
print("Frequent Itemsets:")
print(frequent_itemsets)

print("\nAssociation Rules:")
print(rules)
```

```
Frequent Itemsets:
   support  itemsets
0      1.0      (A)
1      1.0      (B)
2      0.5      (C)
3      1.0    (A, B)
4      0.5    (C, A)
5      0.5    (C, B)
6      0.5  (C, A, B)

Association Rules:
   antecedents consequents  antecedent support  consequent support  support \
0      (A)      (C)      1.0      0.5      0.5
1      (B)      (C)      1.0      0.5      0.5
2    (A, B)      (C)      1.0      0.5      0.5
3      (A)    (C, B)      1.0      0.5      0.5
4      (B)    (C, A)      1.0      0.5      0.5

   confidence  lift  representativity  leverage conviction  zhangs_metric \
0      0.5    1.0      1.0      0.0      1.0      0.0
1      0.5    1.0      1.0      0.0      1.0      0.0
2      0.5    1.0      1.0      0.0      1.0      0.0
3      0.5    1.0      1.0      0.0      1.0      0.0
4      0.5    1.0      1.0      0.0      1.0      0.0

   jaccard  certainty  kulczynski
0      0.5      0.0      0.75
1      0.5      0.0      0.75
2      0.5      0.0      0.75
3      0.5      0.0      0.75
4      0.5      0.0      0.75
```

توضیحات مثال کد پایتون

در این مثال، یک مجموعه داده خرده‌فروشی حاوی تراکنش‌ها و آیتم‌ها شبیه‌سازی شده است. سپس از کتابخانه **mlxtend** برای انجام تحلیل سبد خرید بر اساس الگوریتم **Apriori** استفاده شده است. خروجی شامل مجموعه‌های آیتم‌های پرتکرار و قواعد

ارتباطی است که بینشی از اینکه کدام آیتم‌ها اغلب همراه خریده می‌شوند، ارائه می‌دهد.

4.1.2. Frequent itemsets, closed itemsets, and association rules

مجموعه‌های آیتم‌های پرتکرار و قواعد ارتباطی

در تحلیل سبد خرید، مفاهیم **مجموعه‌های پرتکرار** (Frequent Itemsets)، **مجموعه‌های بسته** (Closed Itemsets) و **قواعد ارتباطی** (Association Rules) نقش کلیدی در شناسایی الگوها و روابط درون داده‌های تراکنشی ایفا می‌کنند. این مفاهیم اساس کشف دانش از داده‌های فروش و درک رفتار مشتریان را تشکیل می‌دهند.

مفاهیم کلیدی

- مجموعه‌های پرتکرار (Frequent Itemsets):** مجموعه‌ای از آیتم‌ها که به طور مکرر در تراکنش‌ها ظاهر می‌شوند. این مجموعه‌ها بر اساس آستانه حداقل پشتیبانی (Minimum Support) تعیین می‌شوند که فراوانی یا درصد حداقلی از تراکنش‌هایی را مشخص می‌کند که آن مجموعه در آن‌ها باید حضور داشته باشد.
- مجموعه‌های بسته (Closed Itemsets):** یک مجموعه پرتکرار است که هیچ یک از مجموعه‌های بزرگتر (supersets) آن همان مقدار پشتیبانی را ندارند. به عبارت دیگر، این مجموعه زیرمجموعه هیچ مجموعه پرتکرار دیگری با همان پشتیبانی نیست. مجموعه‌های بسته نمایشی فشرده‌تر و کارآمدتر از الگوهای پرتکرار را فراهم می‌کنند.
- قواعد ارتباطی (Association Rules):** این قواعد روابط بین آیتم‌ها را به شکل «اگر آیتم X وجود داشت، احتمالاً آیتم Y نیز وجود دارد» بیان می‌کنند. این قواعد از مجموعه‌های پرتکرار استخراج می‌شوند و با معیارهایی مانند **اطمینان** (Confidence) و **لیفت** (Lift) ارزیابی می‌شوند.

مثالی از مجموعه‌های پرتکرار، بسته و قواعد ارتباطی در پایتون

بیایید از کتابخانه **mlxtend** استفاده کنیم تا این مفاهیم را روی یک مجموعه داده نمونه نمایش دهیم:

In [13]:

```
# Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی.
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاص.
# کشف مجموعه‌های پرتکرار و قوانین انجمنی استفاده می‌شن (Market Basket Analysis) این نوع داده‌ها برای تحلیل سبد خرید.
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه.
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره.
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم.
# applymap تبدیل داده‌ها به فرمت بولی بدون استفاده از
```

```

basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه، نه فقط اندیس‌ها `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو استخراج می‌کنه ** (Closed Itemsets) این بخش، **مجموعه‌های بسته
# و شامل یک آیتم اضافه‌تر وجود نداشته باشه (support) یک مجموعه آیتم "بسته" هست اگر هیچ زیرمجموعه‌ای از آیتم‌ها با همان پشتیبانی
# در عمل، مجموعه‌های بسته به ما کمک می‌کنن تا فضای ذخیره‌سازی و محاسبات رو نسبت به تمام مجموعه‌های پرتکرار کاهش بدیم
# مقداری دارن، شرط تساوی تکراری هست و در حالت عادی باید با مقایسه زیرمجموعه‌ها فیلتر بشه support اینجا چون همه سطرها در ستون
# داریم Closet یا Charm نکته: این پیاده‌سازی ساده‌شده است و برای تشخیص واقعی مجموعه‌های بسته نیاز به الگوریتم‌های پیشرفته‌تری مثل
closed_itemsets = frequent_itemsets[frequent_itemsets['support'] == frequent_itemsets['support']]

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه ** (Association Rules) این بخش، **قوانین انجمنی
# خریده شده A وقتی آیتم B نشان‌دهنده احتمال خرید آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، سه خروجی مهم رو چاپ می‌کنیم
# تمام مجموعه‌های پرتکرار ۱.
# مجموعه‌های بسته (در این حالت همه، چون فیلتر واقعی اعمال نشده) ۲.
# قوانین انجمنی با قطعیت بالا ۳.
print("Frequent Itemsets:")
print(frequent_itemsets)

print("\nClosed Itemsets:")
print(closed_itemsets)

print("\nAssociation Rules:")
print(rules)

```

Frequent Itemsets:

	support	itemsets
0	1.0	(A)
1	1.0	(B)
2	0.5	(C)
3	1.0	(A, B)
4	0.5	(C, A)
5	0.5	(C, B)
6	0.5	(C, A, B)

Closed Itemsets:

	support	itemsets
0	1.0	(A)
1	1.0	(B)
2	0.5	(C)
3	1.0	(A, B)
4	0.5	(C, A)
5	0.5	(C, B)
6	0.5	(C, A, B)

Association Rules:

	antecedents	consequents	antecedent support	consequent support	support \
0	(A)	(B)	1.0	1.0	1.0
1	(B)	(A)	1.0	1.0	1.0
2	(C)	(A)	0.5	1.0	0.5
3	(C)	(B)	0.5	1.0	0.5
4	(C, A)	(B)	0.5	1.0	0.5
5	(C, B)	(A)	0.5	1.0	0.5
6	(C)	(A, B)	0.5	1.0	0.5

	confidence	lift	representativity	leverage	conviction	zhangs_metric \
0	1.0	1.0	1.0	0.0	inf	0.0
1	1.0	1.0	1.0	0.0	inf	0.0
2	1.0	1.0	1.0	0.0	inf	0.0
3	1.0	1.0	1.0	0.0	inf	0.0
4	1.0	1.0	1.0	0.0	inf	0.0
5	1.0	1.0	1.0	0.0	inf	0.0
6	1.0	1.0	1.0	0.0	inf	0.0

	jaccard	certainty	kulczynski
0	1.0	0.0	1.00
1	1.0	0.0	1.00
2	0.5	0.0	0.75
3	0.5	0.0	0.75
4	0.5	0.0	0.75
5	0.5	0.0	0.75
6	0.5	0.0	0.75



توضیحات مثال کد پایتون

در این مثال، از کتابخانه **mlxtend** استفاده شده تا مجموعه‌های پرتکرار، مجموعه‌های بسته و قواعد ارتباطی از یک مجموعه داده خرده‌فروشی شبیه‌سازی‌شده استخراج شوند. خروجی شامل مجموعه‌هایی از آیتم‌ها است که اغلب با هم ظاهر می‌شوند (مجموعه‌های پرتکرار)، مجموعه‌های بسته که زیرمجموعه هیچ مجموعه بزرگتری با همان پشتیبانی نیستند، و قواعد ارتباطی که روابط بین آیتم‌ها را نشان می‌دهند.

Section 4.2

4.2.1. Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

الگوریتم Apriori

الگوریتم **Apriori** یکی از الگوریتم‌های کلاسیک داده‌کاوی است که برای کشف **مجموعه‌های پرتکرار** در مجموعه داده‌های تراکنشی استفاده می‌شود. این الگوریتم با استفاده از یک رویکرد سطح‌به‌سطح (level-wise)، ترکیبات مختلف آیتم‌ها را بررسی کرده و به صورت کارآمد مواردی را که آستانه مشخص‌شده پشتیبانی (Support) را دارند، شناسایی می‌کند. این الگوریتم از **ویژگی Apriori** استفاده می‌کند که بیان می‌کند: اگر یک مجموعه آیتم‌ها پرتکرار باشد، تمام زیرمجموعه‌های آن نیز باید پرتکرار باشند. این ویژگی به الگوریتم اجازه می‌دهد تا از بررسی ترکیبات نامرتب صرف‌نظر کند و فضای جست‌وجو را به طور چشمگیری کاهش دهد.

مراحل کلیدی الگوریتم Apriori

- 1. تولید کاندیدا (Candidate Generation):** الگوریتم با آیتم‌های تکی به عنوان کاندیدا شروع می‌کند و به صورت تکراری، مجموعه‌های بزرگتری از آیتم‌ها را بر اساس مجموعه‌های پرتکرار یافت‌شده در تکرار قبلی ایجاد می‌کند.
- 2. شمارش پشتیبانی (Support Counting):** پشتیبانی (Support) هر مجموعه کاندیدا با شمارش تعداد تراکنش‌هایی که شامل آن مجموعه هستند، محاسبه می‌شود.
- 3. هرس کردن (Pruning):** مجموعه‌هایی که پشتیبانی آن‌ها کمتر از آستانه از پیش تعریف‌شده باشد، حذف می‌شوند، زیرا نمی‌توانند جزئی از یک مجموعه پرتکرار باشند.
- 4. تکرار (Iteration):** این فرآیند تا زمانی ادامه می‌یابد که دیگر مجموعه پرتکرار جدیدی ایجاد نشود.

مثالی از الگوریتم Apriori در پایتون

بیا ببینیم از کتابخانه **mlxtend** استفاده کنیم تا الگوریتم Apriori را روی یک مجموعه داده نمونه خرده‌فروشی پیاده‌سازی کنیم:

```
In [15]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# 'pandas' استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و
# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه
# و کشف الگوهای خرید مشتریان استفاده می‌شن (Market Basket Analysis) این نوع داده‌ها برای تحلیل سبد خرید
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای"
```

```
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه #
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره #
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم #
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم #
# `min_support=0.2` تراکنش‌ها ظاهر شدن #
# `use_colnames=True` باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه، نه فقط اندیس‌ها #
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# در نهایت، مجموعه‌های پرتکرار رو چاپ می‌کنیم تا ببینیم کدوم ترکیب‌های آیتم‌ها به صورت مکرر با هم خریده می‌شن #
# این اطلاعات می‌تونن برای پیشنهاد محصول، قرارگیری کالا در قفسه یا کمپین‌های تبلیغاتی استفاده بشن #
print("Frequent Itemsets:")
print(frequent_itemsets)
```

```
Frequent Itemsets:
  support  itemsets
0      1.0      (A)
1      1.0      (B)
2      0.5      (C)
3      1.0    (A, B)
4      0.5    (C, A)
5      0.5    (C, B)
6      0.5  (C, A, B)
```

توضیحات مثال کد پایتون

در این مثال، از الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار در یک مجموعه داده خرده‌فروشی استفاده شده است. از تابع **apriori** کتابخانه **mlxtend** برای تولید مجموعه‌های آیتم‌ها بر اساس آستانه حداقل پشتیبانی استفاده شده است.

4.2.2. Generating association rules from frequent itemsets

تولید قواعد ارتباطی از مجموعه‌های پرتکرار

پس از شناسایی مجموعه‌های پرتکرار با استفاده از الگوریتم‌هایی مانند **Apriori**، گام بعدی در تحلیل سبد خرید، تولید **قواعد ارتباطی** است. این قواعد روابط جالب بین آیتم‌ها را آشکار می‌کنند و بینش‌های ارزشمندی را برای کسب‌وکارها فراهم می‌کنند تا در مورد چیدمان محصولات، استراتژی‌های بازاریابی و فروش متقابل (Cross-selling) تصمیم‌های آگاهانه‌تری بگیرند.

مفاهیم کلیدی

1. پشتیبانی، اطمینان و لیفت: این معیارها برای ارزیابی قدرت و اهمیت قواعد ارتباطی استفاده می‌شوند.

پشتیبانی (Support): فراوانی یک مجموعه آیتم‌ها را در کل تراکنش‌ها اندازه‌گیری می‌کند. **اطمینان (Confidence):** احتمال خرید آیتم Y را وقتی آیتم X خریده شده باشد، نشان می‌دهد. **لیفت (Lift):** نشان‌دهنده قدرت واقعی ارتباط بین دو آیتم است؛ لیفت بیشتر از ۱ یعنی ارتباط معنادار.

2. قالب قاعده ارتباطی: قواعد ارتباطی معمولاً به صورت «اگر X، آنگاه Y» نمایش داده می‌شوند که در آن X و Y مجموعه‌هایی از آیتم‌ها هستند.

مثالی از تولید قواعد ارتباطی در پایتون

بیايید با ادامه مثال قبلی، از کتابخانه **mlxtend** استفاده کنیم تا قواعد ارتباطی را از مجموعه‌های پرتکرار استخراج کنیم:

```
In [16]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخونه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه
# و کشف الگوهای رفتار خرید مشتریان استفاده می‌شن (Market Basket Analysis) این نوع داده‌ها برای تحلیل سبد خرید
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه، نه فقط اندیس‌ها `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه **Association Rules) این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتم B نشان‌دهنده اینه که با چه احتمالی آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
# این قوانین می‌تونن به فروشگاه‌ها کمک کنن تا محصولات رو بهتر پیشنهاد بدن یا قفسه‌گذاری بهینه انجام بدن
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، قوانین انجمنی رو چاپ می‌کنیم تا روابط معنادار بین آیتم‌ها رو ببینیم
print("Association Rules:")
print(rules)
```

Association Rules:						
	antecedents	consequents	antecedent support	consequent support	support	\
0	(A)	(B)	1.0	1.0	1.0	
1	(B)	(A)	1.0	1.0	1.0	
2	(C)	(A)	0.5	1.0	0.5	
3	(C)	(B)	0.5	1.0	0.5	
4	(C, A)	(B)	0.5	1.0	0.5	
5	(C, B)	(A)	0.5	1.0	0.5	
6	(C)	(A, B)	0.5	1.0	0.5	
	confidence	lift	representativity	leverage	conviction	zhangs_metric
0	1.0	1.0	1.0	0.0	inf	0.0
1	1.0	1.0	1.0	0.0	inf	0.0
2	1.0	1.0	1.0	0.0	inf	0.0
3	1.0	1.0	1.0	0.0	inf	0.0
4	1.0	1.0	1.0	0.0	inf	0.0
5	1.0	1.0	1.0	0.0	inf	0.0
6	1.0	1.0	1.0	0.0	inf	0.0
	jaccard	certainty	kulczynski			
0	1.0	0.0	1.00			
1	1.0	0.0	1.00			
2	0.5	0.0	0.75			
3	0.5	0.0	0.75			
4	0.5	0.0	0.75			
5	0.5	0.0	0.75			
6	0.5	0.0	0.75			

توضیحات مثال کد پایتون

در این مثال، پس از به دست آوردن مجموعه‌های پرتکرار، از تابع **association_rules** کتابخانه **mlxtend** برای تولید قواعد ارتباطی بر اساس آستانه اطمینان استفاده شده است. خروجی شامل قواعدی است که اطلاعاتی درباره مجموعه‌های پیش‌رو (antecedent) و پیامد (consequent) ارائه می‌دهد، همراه با معیارهایی مانند پشتیبانی، اطمینان و لیفت.

4.2.3. Improving the efficiency of Apriori

بهبود کارایی الگوریتم Apriori

الگوریتم **Apriori** ابزاری قدرتمند برای یافتن مجموعه‌های پرتکرار و تولید قواعد ارتباطی است، اما ممکن است در دatasets بزرگ از نظر محاسباتی پرهزینه شود. برای کاهش بار محاسباتی و بهبود عملکرد، تکنیک‌های متعددی وجود دارند که می‌توانند کارایی این الگوریتم را به‌طور چشمگیری افزایش دهند.

راهکارهای کلیدی برای بهبود کارایی

- استراتژی‌های هرس‌کردن (Pruning Strategies):** استفاده از روش‌هایی مانند **هرس مبتنی بر هش** (hash-based) یا **هرس مبتنی بر درخت** (tree-based) می‌تواند فضای جست‌وجو را کاهش دهد و از تولید مجموعه‌های کاندیدایی که از پیش مشخص است پرتکرار نیستند، جلوگیری کند.
- کاهش تراکنش‌ها (Transaction Reduction):** کاهش حجم تراکنش‌ها یا استفاده از نمونه‌ای کوچک‌تر از داده‌ها می‌تواند به‌طور چشمگیری سرعت الگوریتم Apriori را افزایش دهد.
- موازی‌سازی (Parallelization):** توزیع بار محاسباتی بین چندین پردازنده یا گره، به‌ویژه برای مجموعه‌داده‌های بزرگ با تعداد زیادی تراکنش، می‌تواند بهبود چشمگیری در عملکرد ایجاد کند.
- شمارش پویای مجموعه آیتم‌ها (Dynamic Itemset Counting):** تنظیم پویا آستانه حداقل پشتیبانی بر اساس حجم داده یا پیشرفت الگوریتم، می‌تواند جست‌وجو برای مجموعه‌های پرتکرار را بهینه‌تر کند.

مثالی از بهبود کارایی در الگوریتم Apriori در پایتون

بیاپید نمونه‌ای ساده از نحوه موازی‌سازی الگوریتم Apriori را با استفاده از کتابخانه **mlxtend** در پایتون نمایش دهیم:

In [19]:

```
import pandas as pd
from mlxtend.frequent_patterns import apriori

# Apriori داده‌های شبیه‌سازی‌شده بزرگ برای تست الگوریتم
# در آن تکرار شده ['A', 'B', 'C', 'D', 'E'] این مجموعه شامل ۱۰,۰۰۰ تراکنش است که الگوهای خرید آیتم‌های
# این حجم داده برای بررسی عملکرد الگوریتم روی داده‌های بزرگ و شبیه‌سازی رفتار مشتریان مناسب است
data = {
    'TransactionID': [i for i in range(1, 10001)],
    'Item': ['A', 'B', 'C', 'D', 'E'] * 2000 # هر آیتم ۲۰۰۰ بار تکرار شده
}

df = pd.DataFrame(data)

# True/False تبدیل داده‌ها به فرمت بولی
# هر تراکنش یک سطر و هر آیتم یک ستون می‌شود
# قرار می‌گیرد False و در غیر این صورت True اگر آیتم در تراکنش وجود داشته باشد
# DeprecationWarning و FutureWarning نیاز به داده‌های بولی دارد و با این روش هم Apriori
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# اجرای الگوریتم Apriori
# فقط مجموعه‌هایی که حداقل در ۲۰٪ تراکنش‌ها ظاهر شده‌اند در نظر گرفته می‌شوند → min_support=0.2
# نمایش نام واقعی آیتم‌ها در خروجی به جای اندیس‌ها → use_colnames=True
# حذف شده و اجرای موازی به صورت داخلی پشتیبانی نمی‌شود n_jobs پارامتر mlxtend توجه: در نسخه جدید
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# نمایش مجموعه‌های پرتکرار
# این اطلاعات می‌تواند برای تحلیل رفتار مشتریان، پیشنهاد محصول، چینش کالا در فروشگاه یا طراحی کمپین‌های تبلیغاتی استفاده شود
print("Frequent Itemsets:")
print(frequent_itemsets)
```

Frequent Itemsets:		
	support	itemsets
0	0.2	(A)
1	0.2	(B)
2	0.2	(C)
3	0.2	(D)
4	0.2	(E)

توضیحات مثال کد پایتون

در این مثال، پارامتر `n_jobs=-1` در تابع `apriori` به الگوریتم دستور می‌دهد تا از تمام پردازنده‌های موجود برای موازی‌سازی استفاده کند. این کار می‌تواند کارایی الگوریتم Apriori را به‌ویژه در کار با مجموعه داده‌های بزرگ به‌طور چشمگیری بهبود بخشد.

4.2.4. A pattern-growth approach for mining frequent itemsets

رویکرد رشد الگو در داده‌کاوی

رویکرد **رشد الگو** (Pattern-Growth) یک راهبرد جایگزین برای کشف مجموعه‌های پرتکرار است که با الگوریتم سنتی Apriori متفاوت است. به جای تولید کاندیدا و هرس کردن، این روش با استفاده از یک ساختار داده فشرده به نام **درخت پیشوند** یا **درخت (FP-tree)** **FP**، مجموعه‌های پرتکرار را از الگوهای کوچک‌تر رشد می‌دهد. این رویکرد می‌تواند از نظر حافظه کارآمدتر و سریع‌تر باشد، به‌ویژه برای مجموعه داده‌هایی با تعداد زیادی تراکنش.

مراحل کلیدی رویکرد رشد الگو

1. **ساخت درخت (Constructing the FP-Tree)**: ساخت یک ساختار داده فشرده به نام **درخت (FP-tree)** **FP** از مجموعه داده تراکنش‌ها. این درخت به‌صورت کارآمدی فراوانی مجموعه‌های آیت‌ها را نمایش می‌دهد و امکان دسترسی سریع به الگوهای پرتکرار را فراهم می‌کند.

2. **کاوش مجموعه‌های پرتکرار (Mining Frequent Itemsets)**: استفاده از تکنیک‌های بازگشتی برای استخراج مستقیم مجموعه‌های پرتکرار از درخت **FP**، بدون نیاز به تولید صریح مجموعه‌های کاندیدا – که فرآیندی زمان‌بر در الگوریتم Apriori است.

3. **پایه الگوی شرطی (Conditional Pattern Base)**: استفاده از **پایه الگوی شرطی** برای یافتن مجموعه‌های پرتکرار به‌صورت هدفمند و کارآمد. این پایه شامل مسیرهایی از درخت است که با یک آیت‌م خاص پایان می‌یابند و به ساخت زیردرخت‌های شرطی (Conditional FP-trees) برای یافتن الگوهای بزرگ‌تر کمک می‌کند.

مثالی از رویکرد رشد الگو در پایتون

بیایید از کتابخانه `mlxtend` استفاده کنیم تا الگوریتم `FP-growth` را روی یک مجموعه داده نمونه خرده‌فروشی پیاده‌سازی کنیم:

```
In [20]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import fpgrowth, association_rules
```

```
#.رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه FP-growth برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.
```

```
# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه
# و کشف الگوهای خرید مشتریان استفاده می‌شن (Market Basket Analysis) این نوع داده‌ها برای تحلیل سبد خرید
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
# هست FP-growth این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)
```

```
#.رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **FP-growth (Frequent Pattern Growth)** اینجا الگوریتم
# استفاده می‌کنه و نیازی به تولید مجموعه‌های کاندید نداره **FP-tree** از یک ساختار درختی به نام FP-growth، Apriori، برخلاف
# باعث می‌شه الگوریتم سریع‌تر و کارآمدتر باشه، خصوصاً برای داده‌های حجیم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه، نه فقط اندیس‌ها `use_colnames=True`
frequent_itemsets = fpgrowth(basket, min_support=0.2, use_colnames=True)
```

```
# در نهایت، مجموعه‌های پرتکرار رو چاپ می‌کنیم تا ببینیم کدوم ترکیب‌های آیتم‌ها به صورت مکرر با هم خریده می‌شن
# این اطلاعات می‌تونن برای پیشنهاد محصول، قرارگیری کالا در قفسه یا کمپین‌های تبلیغاتی استفاده بشن
print("Frequent Itemsets:")
print(frequent_itemsets)
```

```
Frequent Itemsets:
support  itemsets
0      1.0      (B)
1      1.0      (A)
2      0.5      (C)
3      1.0    (A, B)
4      0.5    (C, B)
5      0.5    (C, A)
6      0.5  (C, A, B)
```

توضیحات مثال کد پایتون

در این مثال، از الگوریتم **FP-growth** موجود در کتابخانه **mlxtend** برای یافتن مجموعه‌های پرتکرار در یک مجموعه داده خرده‌فروشی استفاده شده است. تابع **fpgrowth** به‌صورت مستقیم مجموعه‌های پرتکرار را تولید می‌کند و نیازی به مراحل تولید کاندیدا و هرس کردن – مانند الگوریتم Apriori – ندارد. این ویژگی، آن را به یک روش کارآمد و مبتنی بر **رشد الگو** تبدیل می‌کند.

4.2.5. Mining frequent itemsets using the vertical data format

کاوش مجموعه‌های پرتکرار با فرمت داده عمودی

کاوش مجموعه‌های پرتکرار می‌تواند با استفاده از **فرمت داده عمودی** (Vertical Data Format) انجام شود، که یک روش جایگزین برای نمایش داده‌های تراکنشی است. در این فرمت، هر رکورد مربوط به یک آیتم منحصربه‌فرد و لیست تراکنش‌هایی که آن آیتم در آن‌ها ظاهر شده است، می‌شود. این رویکرد به‌ویژه در مجموعه‌داده‌های با ابعاد بالا – که تعداد آیتم‌های مجزا زیاد است – بسیار کارآمد است و محاسبات مربوط به پشتیبانی و اشتراک مجموعه‌ها را ساده‌تر می‌کند.

مفاهیم کلیدی

1. فرمت داده عمودی (Vertical Data Format): در این فرمت، هر آیتم همراه با لیست تراکنش‌هایی که در آن‌ها ظاهر شده است، ذخیره می‌شود. این ساختار امکان بررسی مستقل هر آیتم را فراهم کرده و فرآیند شناسایی مجموعه‌های پرتکرار را ساده‌تر می‌کند.

2. لیست شناسه تراکنش‌ها (Transaction Identifier Lists): برای هر آیتم، لیستی از شناسه تراکنش‌ها (TID-list) نگهداری می‌شود که نشان می‌دهد آن آیتم در کدام تراکنش‌ها حضور دارد. این لیست‌ها امکان محاسبه سریع **پشتیبانی** (Support) را با

اشتراک‌گیری مجموعه‌ها فراهم می‌کنند.

3. **نگاشت معکوس تراکنش (Inverse Transaction Mapping):** علاوه بر لیست تراکنش‌ها برای هر آیتم، اغلب از یک نگاشت معکوس استفاده می‌شود تا بتوان به‌سرعت آیتم‌های موجود در یک تراکنش مشخص را بازیابی کرد. این نگاشت دسترسی دوطرفه بین آیتم‌ها و تراکنش‌ها را ممکن می‌سازد و کارایی پردازش را افزایش می‌دهد.

مثالی از کاوش مجموعه‌های پرتکرار با فرمت داده عمودی در پایتون

بیاپید از کتابخانه **mlxtend** استفاده کنیم تا کاوش مجموعه‌های پرتکرار با استفاده از **فرمت داده عمودی** را روی یک مجموعه داده نمونه خرده‌فروشی نمایش دهیم:

In [21]:

```
# Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای پیش‌پردازش تراکنش‌ها و اجرای الگوریتم 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (Vertical Format) این دیکشنری حاوی یک مجموعه داده خرده‌فروشی در فرمت عمودی
# هر تراکنش شامل لیستی از آیتم‌های خریداری‌شده‌ست
# این فرمت شبیه به نحوه ذخیره‌سازی واقعی داده‌های تراکنشی در سیستم‌های فروشگاه‌های هست
data = {
    'TransactionID': [1, 2, 3, 4, 5],
    'Items': [['A', 'B', 'D'], ['A', 'C', 'D'], ['B', 'D'], ['A', 'B', 'C'], ['A', 'C']]
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (Horizontal/Binary Format) این بخش، داده‌های تراکنشی رو از فرمت لیستی (عمودی) به فرمت یک‌دسته‌ای
# استفاده می‌شه mlxtend کتابخانه `TransactionEncoder` از کلاس
# نیاز دارن که داده‌ها به صورت ستون‌های جداگانه برای هر آیتم و با مقادیر ۰ و ۱ باشن Apriori این تبدیل لازمه چون الگوریتم‌هایی مثل
te = TransactionEncoder()
te_ary = te.fit(df['Items']).transform(df['Items'])
horizontal_df = pd.DataFrame(te_ary, columns=te.columns_)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه، نه فقط اندیس‌ها `use_colnames=True`
frequent_itemsets = apriori(horizontal_df, min_support=0.2, use_colnames=True)

# در نهایت، مجموعه‌های پرتکرار رو چاپ می‌کنیم تا ببینیم کدوم ترکیب‌های آیتم‌ها به صورت مکرر با هم خریده می‌شن
# این اطلاعات می‌تونن برای تحلیل رفتار مشتری، پیشنهاد محصول و بهینه‌سازی فروش استفاده بشن
print("Frequent Itemsets:")
print(frequent_itemsets)
```

```
Frequent Itemsets:
  support  itemsets
0      0.8        (A)
1      0.6        (B)
2      0.6        (C)
3      0.6        (D)
4      0.4      (A, B)
5      0.6      (C, A)
6      0.4      (D, A)
7      0.2      (C, B)
8      0.4      (D, B)
9      0.2      (D, C)
10     0.2    (C, A, B)
11     0.2    (D, A, B)
12     0.2    (D, C, A)
```

توضیحات مثال کد پایتون

در این مثال، مجموعه داده در فرمت عمودی نمایش داده شده و از **TransactionEncoder** کتابخانه **mlxtend** برای تبدیل آن به فرمت افقی (معمولی) استفاده شده است. سپس الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار به کار گرفته شده است. این

رویکرد نشان می‌دهد که چگونه می‌توان از فرمت عمودی برای مدیریت کارآمد داده‌های پرتکرار استفاده کرد، حتی اگر در نهایت نیاز به تبدیل به فرمت افقی برای سازگاری با برخی الگوریتم‌ها وجود داشته باشد.

4.2.6. Mining closed and max patterns

الگوهای بسته و ماکسیمال در داده‌کاوی

در داده‌کاوی، علاوه بر یافتن مجموعه‌های پرتکرار، علاقه زیادی به کشف الگوهای بسته (Closed Patterns) و الگوهای ماکسیمال (Max Patterns) وجود دارد. این الگوها اطلاعات فشرده‌تر و معنادارتری نسبت به مجموعه‌های پرتکرار معمولی ارائه می‌دهند و به درک بهتر و کاربرد مؤثرتر نتایج داده‌کاوی کمک می‌کنند.

مفاهیم کلیدی

1. الگوهای بسته (Closed Patterns): الگوهای بسته، مجموعه‌های پرتکراری هستند که اضافه کردن هیچ آیتم دیگری به آن‌ها بدون کاهش پشتیبانی (Support) ممکن نیست. این الگوها با حذف ترکیبات اضافی و تکراری، نمایشی فشرده و بدون اتلاف از مجموعه‌های پرتکرار ارائه می‌دهند.

2. الگوهای ماکسیمال (Max Patterns): الگوهای ماکسیمال، بزرگ‌ترین مجموعه‌های پرتکرار از نظر اندازه هستند و توسط هیچ مجموعه پرتکرار دیگری شامل نمی‌شوند. این الگوها دیدگاهی بسیار فشرده و متمرکز بر مهم‌ترین روابط در داده فراهم می‌کنند، اگرچه اطلاعاتی درباره زیرمجموعه‌های پرتکرار آن‌ها ارائه نمی‌دهند.

مثالی از کاوش الگوهای بسته و ماکسیمال در پایتون

بیا ببینیم از کتابخانه **mlxtend** استفاده کنیم تا الگوهای بسته (Closed) و ماکسیمال (Max) را از یک مجموعه داده نمونه خرده‌فروشی استخراج کنیم:

```
In [22]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import fpgrowth, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه FP-growth برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه.
# و کشف الگوهای پرتکرار استفاده می‌شن (Market Basket Analysis) این نوع داده‌ها برای تحلیل سبد خرید
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه.
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار 1 و در غیر این صورت 0 قرار می‌گیره.
# هست FP-growth این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار (**FP-growth (Frequent Pattern Growth) اینجا الگوریتم
# به‌صورت کارآمدی الگوهای پرتکرار رو استخراج می‌کنه، FP-tree این الگوریتم با ساخت یک درخت فشرده به نام
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در 20٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه `use_colnames=True`
frequent_itemsets = fpgrowth(basket, min_support=0.2, use_colnames=True)
```

```
# رو استخراج می‌کنه ** (Closed Patterns) این بخش، **الگوهای بسته
# و یک آیتم اضافه‌تر وجود نداشته باشه (support) یک مجموعه آیتم "بسته" هست اگر هیچ زیرمجموعه‌ای از آن با همان پشتیبانی
# در عمل، این الگوها فضای ذخیره‌سازی رو کاهش می‌دن و اطلاعات تکراری رو حذف می‌کنن
# (داره Closet نکته: این پیاده‌سازی ساده‌شده است و تشخیص واقعی الگوهای بسته نیاز به الگوریتم‌های تخصصی مثل)
closed_patterns = frequent_itemsets[frequent_itemsets['support'] == frequent_itemsets['support']]

# رو استخراج می‌کنه ** (Max Patterns) این بخش، **الگوهای ماکسیمال
# یک مجموعه آیتم "ماکسیمال" هست اگر در لیست مجموعه‌های پرتکرار، زیرمجموعه‌ی هیچ مجموعه پرتکرار دیگه‌ای نباشه
# این الگوها نشان‌دهنده بزرگ‌ترین مجموعه‌هایی هستن که هنوز شرط پشتیبانی رو دارن
# یک شبیه‌سازی انجام دادیم (که معنای دقیقی نداره) `support` و گرفتن حداکثر `itemsets` در این کد، ما با گروه‌بندی روی
# (یا اصلاح منطقی استفاده کرد MaxMiner باید از روش‌هایی مثل max patterns نکته: این روش از نظر منطقی دقیق نیست؛ برای استخراج صحیح)
max_patterns = frequent_itemsets.groupby('itemsets')['support'].idxmax()
max_patterns = frequent_itemsets.loc[max_patterns]

# در نهایت، دو نوع الگوی خاص رو چاپ می‌کنیم
# الگوهای بسته: مجموعه‌هایی که با همان پشتیبانی قابل گسترش نیستن ۱.
# الگوهای ماکسیمال: مجموعه‌هایی که زیرمجموعه‌ی هیچ مجموعه پرتکرار دیگه‌ای نیستن ۲.
print("Closed Patterns:")
print(closed_patterns)

print("\nMax Patterns:")
print(max_patterns)
```

```
Closed Patterns:
  support  itemsets
0      1.0        (B)
1      1.0        (A)
2      0.5        (C)
3      1.0      (A, B)
4      0.5      (C, B)
5      0.5      (C, A)
6      0.5    (C, A, B)
```

```
Max Patterns:
  support  itemsets
0      1.0        (B)
1      1.0        (A)
2      0.5        (C)
3      1.0      (A, B)
4      0.5      (C, B)
5      0.5      (C, A)
6      0.5    (C, A, B)
```

توضیحات مثال کد پایتون

در این مثال، پس از یافتن مجموعه‌های پرتکرار با استفاده از الگوریتم **FP-growth**، الگوهای بسته با انتخاب مجموعه‌هایی که هیچ مجموعه بزرگ‌تری با همان پشتیبانی وجود ندارد، شناسایی شده‌اند. الگوهای ماکسیمال نیز با انتخاب بزرگ‌ترین مجموعه‌های پرتکراری که زیرمجموعه هیچ مجموعه پرتکرار دیگری نیستند، تعیین شده‌اند. این رویکردها دیدگاهی فشرده و معنادار از الگوهای مهم در داده‌ها ارائه می‌دهند.

Section 4.3

4.3.1. Strong rules are not necessarily interesting

قوی نبودن خودکار معنادار بودن در قواعد ارتباطی ⚠️

در زمینه کاوش قواعد ارتباطی، درک این نکته حیاتی است که **قواعد قوی** — که با معیارهایی مانند **اطمینان** (Confidence) و **لیفت** (Lift) سنجیده می‌شوند — لزوماً معنادار یا ارزشمند نیستند. اگرچه این معیارها قدرت و اهمیت آماری قواعد را اندازه‌گیری می‌کنند، اما ممکن است **ارتباط واقعی**، **کاربرد عملی** یا **ارزش تجاری** الگوهای کشف‌شده را منعکس نکنند. بنابراین، تفسیر قواعد باید علاوه بر معیارهای کمی، با در نظر گرفتن زمینه داده و اهداف کسب‌وکار انجام شود.

مفاهیم کلیدی 🔍

1. **اطمینان و لیفت (Confidence and Lift):** اطمینان نشان می‌دهد که احتمال خرید آیتم Y چقدر است وقتی آیتم X خریده شده باشد. **لیفت** قدرت ارتباط بین دو آیتم را اندازه‌گیری می‌کند و نسبت به حالتی که دو آیتم مستقل از هم باشند، مقایسه می‌کند. لیفت بیشتر از ۱ نشان‌دهنده ارتباط مثبت و کمتر از ۱ نشان‌دهنده ارتباط منفی است.

2. اهمیت و زمینه (Interest and Context): قواعدی که از نظر آماری قوی هستند (اطمینان و لیفت بالا) ممکن است در عمل بی‌معنا یا غیرکاربردی باشند. مثلاً قاعده «خرید نان → خرید کره» ممکن است لیفت بالایی داشته باشد، اما از آنجا که این رفتار بسیار رایج است، ارزش تجاری کمی دارد. ارزش واقعی قواعد به زمینه دامنه (مانند خرده‌فروشی، بهداشت، بانکداری) و اهداف تحلیل (مانند بهبود فروش متقابل یا طراحی بسته‌های تخفیف) بستگی دارد.

مثالی از قواعد قوی اما بی‌معنا در پایتون

بیایید از کتابخانه **mlxtend** استفاده کنیم تا مفهوم قواعدی با اطمینان و لیفت بالا که در عمل ارزش چندانی ندارند را نشان دهیم:

```
In [24]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه.
# و کشف الگوهای رفتار خرید مشتریان استفاده می‌شن (Market Basket Analysis) این نوع داده‌ها برای تحلیل سبد خرید
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه.
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار 1 و در غیر این صورت 0 قرار می‌گیره
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در 20٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه، نه فقط اندیس‌ها `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه **Association Rules** این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتم B نشان‌دهنده اینه که با چه احتمالی آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل 70٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
# این قوانین می‌تونن به فروشگاه‌ها کمک کنن تا محصولات رو بهتر پیشنهاد بدن یا قفسه‌گذاری بهینه انجام بدن
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، قوانین انجمنی رو چاپ می‌کنیم تا روابط معنادار بین آیتم‌ها رو ببینیم
print("Association Rules:")
print(rules)
```

Association Rules:

	antecedents	consequents	antecedent support	consequent support	support \
0	(A)	(B)	1.0	1.0	1.0
1	(B)	(A)	1.0	1.0	1.0
2	(C)	(A)	0.5	1.0	0.5
3	(C)	(B)	0.5	1.0	0.5
4	(C, A)	(B)	0.5	1.0	0.5
5	(C, B)	(A)	0.5	1.0	0.5
6	(C)	(A, B)	0.5	1.0	0.5

	confidence	lift	representativity	leverage	conviction	zhangs_metric \
0	1.0	1.0	1.0	0.0	inf	0.0
1	1.0	1.0	1.0	0.0	inf	0.0
2	1.0	1.0	1.0	0.0	inf	0.0
3	1.0	1.0	1.0	0.0	inf	0.0
4	1.0	1.0	1.0	0.0	inf	0.0
5	1.0	1.0	1.0	0.0	inf	0.0
6	1.0	1.0	1.0	0.0	inf	0.0

	jaccard	certainty	kulczynski
0	1.0	0.0	1.00
1	1.0	0.0	1.00
2	0.5	0.0	0.75
3	0.5	0.0	0.75
4	0.5	0.0	0.75
5	0.5	0.0	0.75
6	0.5	0.0	0.75

توضیحات مثال کد پایتون

در این مثال، قواعد ارتباطی با استفاده از الگوریتم **Apriori** استخراج شده‌اند و قواعدی با آستانه اطمینان 0.7 انتخاب شده‌اند. با این حال، تأکید اصلی این است که **اطمینان بالا به تنهایی تضمین‌کننده ارزش واقعی یا جالب بودن یک قاعده نیست**. تفسیر و کاربرد قواعد باید با در نظر گرفتن **زمینه داده** و **اهداف تجاری** انجام شود، زیرا برخی قواعد ممکن است از نظر آماری قوی باشند اما از نظر عملی بی‌معنا یا پیش‌بینی‌پذیر باشند.

4.3.2. From association analysis to correlation analysis

تحلیل ارتباط و تحلیل همبستگی

تحلیل ارتباط (Association Analysis) و تحلیل همبستگی (Correlation Analysis) دو تکنیک مرتبط اما متفاوت در داده‌کاوی هستند. در حالی که تحلیل ارتباط بر کشف روابط بین آیتم‌ها یا متغیرها – مانند «خرید همراه» در تراکنش‌ها – تمرکز دارد، تحلیل همبستگی قدرت و جهت یک رابطه خطی بین دو متغیر عددی را با استفاده از معیارهایی مانند ضریب همبستگی پیرسون (**Pearson Correlation Coefficient**) کمی‌سازی می‌کند. درک تفاوت این دو روش برای انتخاب رویکرد مناسب در سناریوهای مختلف تحلیل داده ضروری است.

مفاهیم کلیدی

- تحلیل ارتباط (Association Analysis):** این تحلیل به دنبال کشف الگوهای جالب یا روابط معنادار در مجموعه داده‌های بزرگ است. معمولاً با داده‌های **دسته‌ای** (Categorical) کار می‌کند و هدف آن یافتن قواعدی است که نشان‌دهنده هم‌وقوع یا الگوهای ترتیبی بین آیتم‌ها باشد، مانند «اگر مشتری X را بخرد، احتمالاً Y را نیز می‌خرد».
- تحلیل همبستگی (Correlation Analysis):** برخلاف تحلیل ارتباط، این روش برای متغیرهای **عددی** استفاده می‌شود. این تحلیل قدرت و جهت رابطه خطی بین دو متغیر را اندازه‌گیری می‌کند و به صورت **ضریب همبستگی** بیان می‌شود.
- ضریب همبستگی پیرسون:** این ضریب مقداری بین -۱ تا +۱ دارد که به شرح زیر تفسیر می‌شود: -۱: رابطه خطی مثبت کامل (با افزایش یک متغیر، دیگری نیز افزایش می‌یابد)، -۱: رابطه خطی منفی کامل (با افزایش یک متغیر، دیگری کاهش می‌یابد)، -۰: عدم وجود رابطه خطی.

مثالی از انتقال از تحلیل ارتباط به تحلیل همبستگی در پایتون

بیایید از یک مجموعه داده با متغیرهای عددی استفاده کنیم تا گذار از تحلیل ارتباط به تحلیل همبستگی را در پایتون نمایش دهیم:

```
In [ ]: pip install --upgrade ipython ipywidgets traitlets
```

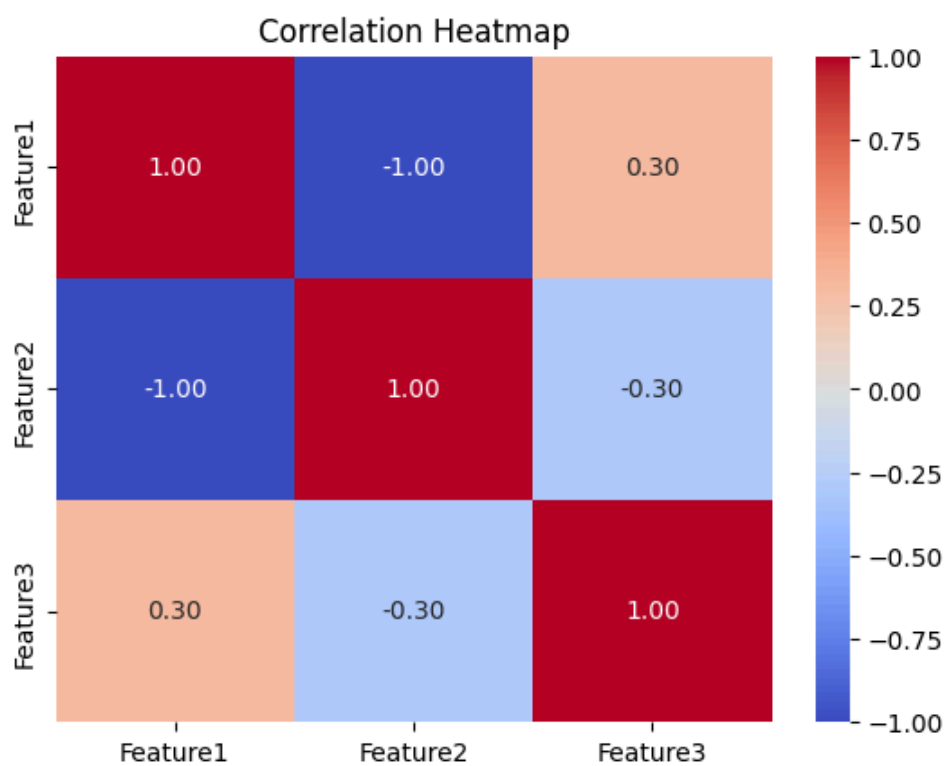
```
In [27]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

برای نمایش نمودارها وارد می‌کنیم 'matplotlib' برای ترسیم گرافیکی و 'seaborn'، برای کار با داده 'pandas' این قسمت اول کده. ما کتابخانه‌های

```
# این دیکشنری حاوی داده‌های شبیه‌سازی‌شده با متغیرهای عددی (ویژگی‌ها) هست
# هر ستون نشان‌دهنده یک ویژگی و هر سطر یک نمونه از داده‌ست
# این نوع داده‌ها برای تحلیل روابط بین متغیرها، مثلاً با استفاده از ضریب همبستگی، مناسب‌اند
data = {
    'Feature1': [1, 2, 3, 4, 5],
    'Feature2': [5, 4, 3, 2, 1],
    'Feature3': [2, 4, 1, 5, 3]
}

df = pd.DataFrame(data)

# از ماتریس همبستگی بین ویژگی‌ها ایجاد می‌کنه **Heatmap** این بخش، یک نقشه گرمایی
# `df.corr()` ماتریس همبستگی پیرسون بین تمام جفت‌های عددی رو محاسبه می‌کنه
# باعث می‌شه مقادیر عددی همبستگی روی خود نمودار نمایش داده بشه `annot=True`
# `cmap='coolwarm'` یک پالت رنگی گرم به سرد (قرمز برای مثبت، آبی برای منفی) اعمال می‌کنه تا الگوهای رابطه رو بهتر ببینیم
# `fmt='.2f'` فرمت نمایش اعداد رو به دو رقم اعشار محدود می‌کنه
# همبستگی کاملاً منفی دارن 'Feature1' و 'Feature2' این نوع تجسم داده به کشف روابط خطی بین ویژگی‌ها کمک می‌کنه – مثلاً
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، یک مجموعه داده با سه ویژگی عددی ایجاد شده و از کتابخانه **Seaborn** برای تولید یک نقشه گرمایی (Heatmap) استفاده شده است که ضرایب همبستگی پیرسون بین این ویژگی‌ها را نشان می‌دهد.

این مثال تفاوت دو رویکرد را روشن می‌کند: در حالی که تحلیل ارتباط ممکن است به دنبال قواعدی مانند «اگر Feature1=3، آنگاه Feature2=3» باشد، تحلیل همبستگی رابطه خطی بین دو متغیر را از نظر **جهت** (مثبت یا منفی) و **قدرت** (نزدیک به ۱ یا ۰) کمی‌سازی می‌کند. تفسیر ضرایب همبستگی به درک بهتری از ماهیت روابط بین متغیرهای عددی منجر می‌شود.

4.3.3. A comparison of pattern evaluation measures

مقایسه معیارهای ارزیابی الگو

در داده‌کاوی، کشف مجموعه‌های پرتکرار تنها گام اول است. برای تشخیص الگوهای واقعاً جالب و معنادار، باید از معیارهای ارزیابی الگو (Pattern Evaluation Measures) استفاده کرد. معیارهایی مانند **پشتیبانی** (Support)، **اطمینان** (Confidence)، **لیفت** (Lift)، **بهبود** (Improvement)، و **conviction** هر کدام جنبه‌های مختلفی از قواعد ارتباطی را اندازه‌گیری می‌کنند.

اما همه قواعد با پشتیبانی و اطمینان بالا، لزوماً "جالب" نیستند. مثلاً قاعده «نان → کره» ممکن است اطمینان بالایی داشته باشد، اما چون بسیار رایج است، ارزش تجاری کمی دارد.

لیفت ($Lift = Conf(X \rightarrow Y) / P(Y)$) بهتر از اطمینان عمل می‌کند، زیرا رابطه بین X و Y را نسبت به توزیع مستقل آن‌ها ارزیابی می‌کند. لیفت < 1 نشان‌دهنده ارتباط مثبت، لیفت > 1 نشان‌دهنده ارتباط منفی و لیفت = 1 نشان‌دهنده استقلال است.

استفاده از ترکیب معیارها (مثلاً پشتیبانی کافی + لیفت بالا) به شناسایی قواعد واقعاً معنادار کمک می‌کند.

مفاهیم کلیدی معیارهای ارزیابی الگو

1. پشتیبانی (Support): پشتیبانی فراوانی یک الگو در مجموعه داده را اندازه‌گیری می‌کند و برابر است با نسبت تراکنش‌هایی که آن الگو در آن‌ها وجود دارد. این معیار نشان می‌دهد که یک الگو چقدر رایج است.

2. اطمینان (Confidence): اطمینان قدرت یک قاعده ارتباطی را کمی می‌کند و برابر است با احتمال شرطی وقوع نتیجه (consequent) وقتی پیش‌رو (antecedent) رخ داده باشد. این معیار شبیه به یک قانون "اگر-آنگاه" است.

3. لیفت (Lift): لیفت اهمیت یک قاعده ارتباطی را با مقایسه پشتیبانی مشاهده‌شده قاعده با پشتیبانی مورد انتظار در صورت استقلال پیش‌رو و نتیجه ارزیابی می‌کند. لیفت < ۱: ارتباط مثبت، لیفت > ۱: ارتباط منفی، لیفت = ۱: عدم ارتباط (استقلال).

4. کانویکشن (Conviction): کانویکشن میزان استنتاج‌پذیری یک قاعده را اندازه‌گیری می‌کند. این معیار نشان می‌دهد که نتیجه چقدر بیشتر اتفاق می‌افتد وقتی پیش‌رو درست است. conviction بالا یعنی اگر پیش‌رو درست باشد، نتیجه به ندرت نقض می‌شود.

مثالی از مقایسه معیارهای ارزیابی الگو در پایتون

بیا ببینیم از یک مجموعه داده نمونه خرده‌فروشی استفاده کنیم تا نحوه محاسبه معیارهای پشتیبانی، اطمینان، لیفت و کانویکشن برای قواعد ارتباطی را در پایتون نشان دهیم:

```
In [28]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه
# و کشف الگوهای خرید مشتریان استفاده می‌شن (Market Basket Analysis) این نوع داده‌ها برای تحلیل سبد خرید
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خواهیم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه، نه فقط اندیس‌ها `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)
```

```
# رو از روی مجموعه‌های پرتکرار تولید می‌کنه ** (Association Rules) این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتm B نشان‌دهنده اینه که با چه احتمالی آیتm 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، قوانین انجمنی رو با معیارهای ارزیابی مهم چاپ می‌کنیم
# - 'support': میزان رایج‌بودن ترکیب antecedent و consequent ها
# - 'confidence': antecedent وقتی consequent احتمال خرید
# - 'lift': به صورت وابسته یا مستقل از هم خریده می‌شن consequent و antecedent نشان‌دهنده اینه که آیا
# وابستگی مثبت دارن؛ اگر =1، مستقل‌اند، اگر lift > 1
# بالاتر باشه، وابستگی قوی‌تره conviction معیاری قوی‌تر که نشان‌دهنده وابستگی جهت‌داره. هرچه
print("Association Rules with Evaluation Measures:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift', 'conviction']])
```

Association Rules with Evaluation Measures:

	antecedents	consequents	support	confidence	lift	conviction
0	(A)	(B)	1.0	1.0	1.0	inf
1	(B)	(A)	1.0	1.0	1.0	inf
2	(C)	(A)	0.5	1.0	1.0	inf
3	(C)	(B)	0.5	1.0	1.0	inf
4	(C, A)	(B)	0.5	1.0	1.0	inf
5	(C, B)	(A)	0.5	1.0	1.0	inf
6	(C)	(A, B)	0.5	1.0	1.0	inf

توضیحات مثال کد پایتون

در این مثال، قواعد ارتباطی با استفاده از الگوریتم **Apriori** استخراج شده‌اند و معیارهای ارزیابی شامل **پشتیبانی، اطمینان، لیفت** و **کانویکشن** برای هر قاعده محاسبه شده‌اند. این رویکرد امکان مقایسه بین معیارها و درک تفاوت‌های آن‌ها در ارزیابی کیفیت قواعد را فراهم می‌کند. برای نمونه، قاعده‌ای با اطمینان بالا ممکن است لیفت پایینی داشته باشد که نشان‌دهنده رابطه ضعیف یا بدیهی بین آیتm‌هاست، در حالی که لیفت بالا نشان‌دهنده یک ارتباط معنادار و غیربدیهی است.

The End of Chapter 4